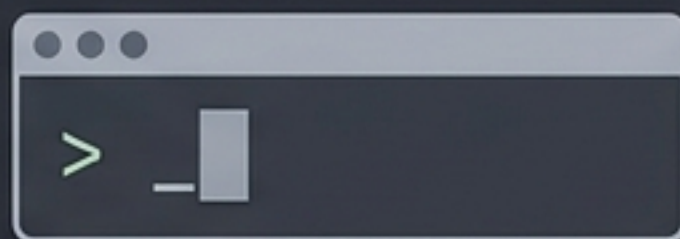


Project > src > TitleSlide.cs

C# Developer's Toolkit

문자열 처리부터 인터랙티브 콘솔 애니메이션까지,
동작하는 애플리케이션을 조립하는 실전 가이드



가장 흔한 실수: 문자열의 불변성 (Immutability)

C#의 모든 문자열 처리 메서드는 자신을 변경하지 않고 새로운 복사본을 반환합니다.



```
static void Main(string[] args)
{
    string input = "Potato Tomato";
    input.ToUpper();
    Console.WriteLine(input);
}
```

원본은 변하지 않음!

실행 결과

```
POTATO TOMATO
potato tomato
```

텍스트 가공 도구 상자 (String Transformation)

대소문자

"Potato Tomato"



ToUpper() / ToLower()



"POTATO TOMATO" /
"potato tomato"

문자열의 영문자를 모두 대문자
또는 소문자로 변환합니다.

공백 제거

" test \n"



Trim()



"test"



문자열 양 옆의
공백 제거

TrimStart() (앞 공백 제거),
TrimEnd() (뒤 공백 제거)

자르기
(배열로)

"감자 고구마 토마토"



Split(' ')



["감자", "고구마",
"토마토"]



특정 문자 또는
문자열로 잘라
배열로 만듭니다.

결합하기
(문자열로)

["감자", "고구마",
"토마토"]



string.Join(",", array)



"감자,고구마,토마토"



배열의 요소를 뭉쳐
문자열로 변경합니다.

시각적 제어 도구 (Console Control Tools)



`Console.Clear()`

콘솔 화면의 모든 글자를
즉시 지웁니다.



`Console.SetCursorPosition(x, y)`

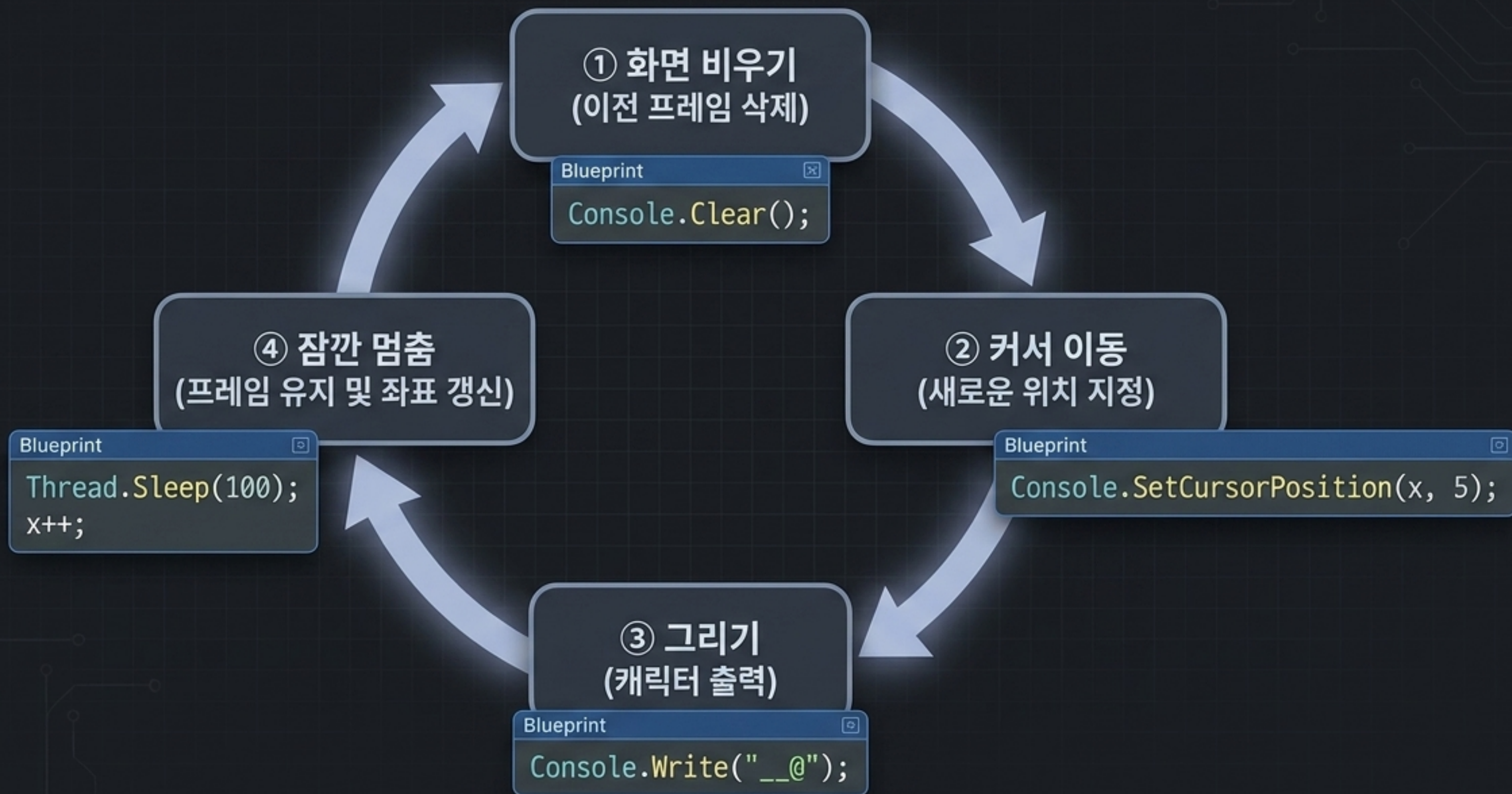
원하는 위치로 출력 커서를
순간이동 시킵니다.



`Thread.Sleep(ms)`

입력한 밀리초(ms) 단위만큼
프로그램의 시간을 멈춥니다.

애니메이션의 원리: 화면 갱신 루프 (Animation Loop)



디테일 렌더링: 꿈틀거리는 달팽이

```
if (x % 3 == 0) Console.Write("__@");  
else if (x % 3 == 1) Console.Write("_^@");  
else Console.Write("^__@");
```

Frame 1 (x=1)

_ ^ @

Frame 2 (x=2)

^ __ @

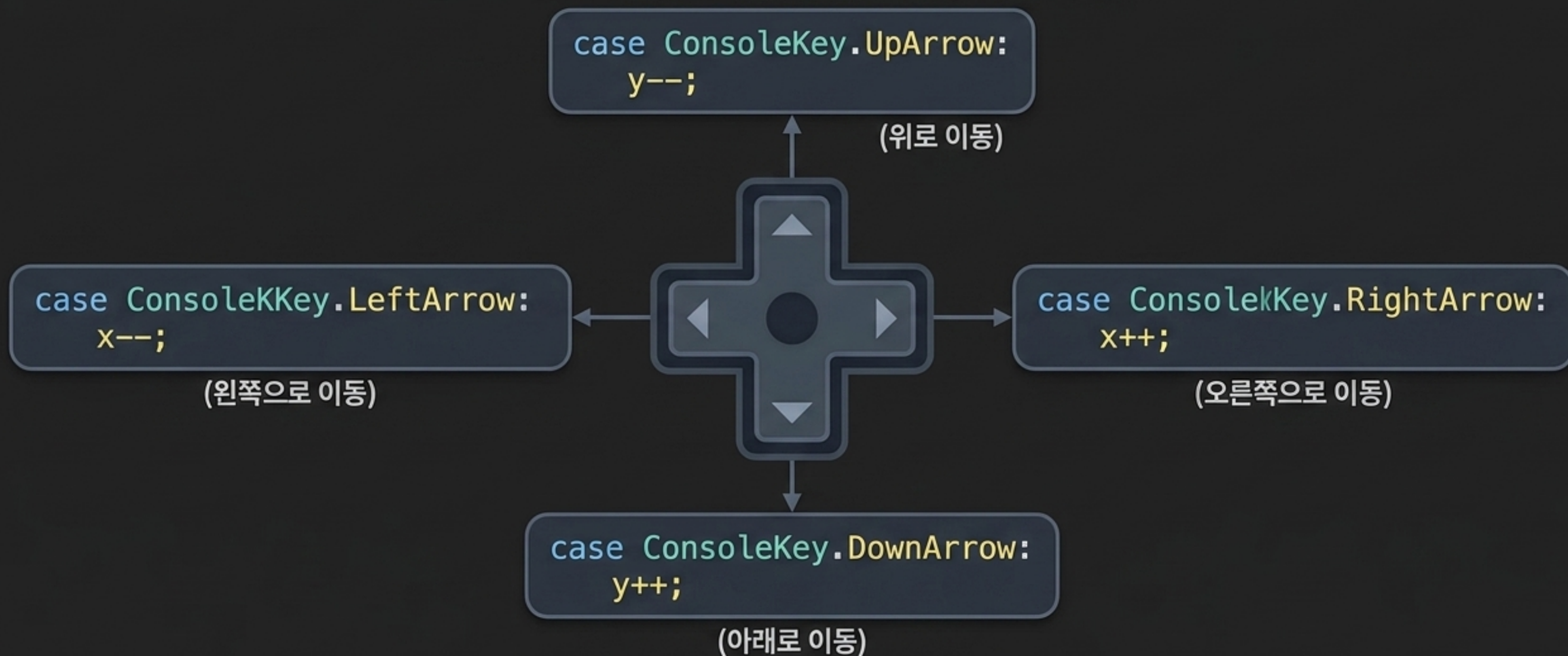
Frame 3 (x=3)

__ @

변수 x의 증가에 따라 달팽이의 모양이
3단계를 거치며 순환합니다.

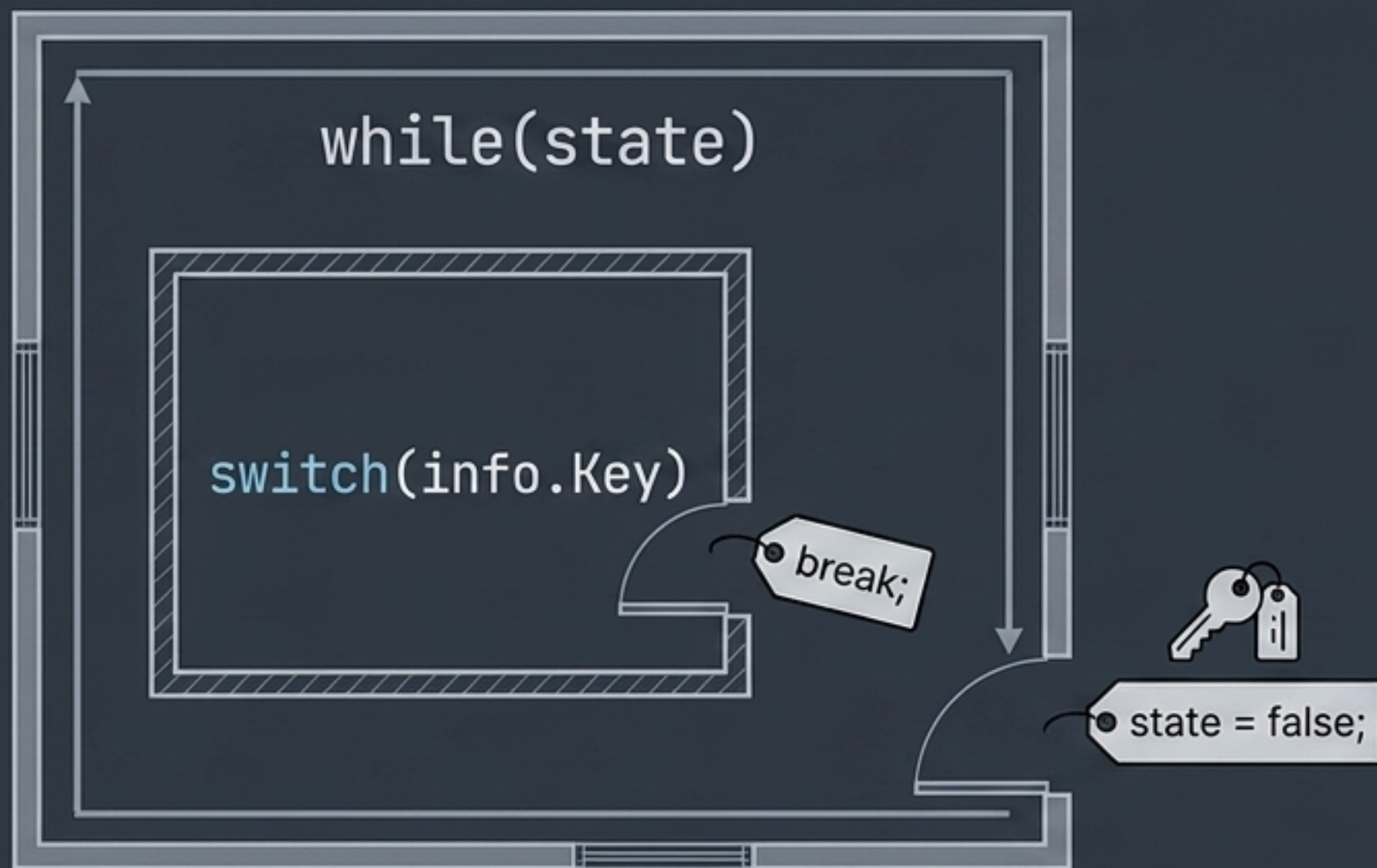
인터랙티브 입력: 방향키로 제어하기

`while (true)` 무한 반복문 안에서 `Console.ReadKey()`로 입력을 대기합니다.



아키텍처: 탈출 스코프 (Escape Scope)

switch 문 내부의 break는 조건문(안쪽 방)만 빠져나갑니다. 무한 반복문(전체 집)을 완전히 종료하려면 제어 변수(bool state)의 상태를 변경해야 합니다.



```
case ConsonkeyKey.X:  
    state = false; // 마스터키 작동  
    break;        // 방문 열기
```

Pro-Tip: 화살표(-->) 연산자의 착시 효과

C#에는 --> 라는 화살표 연산자가 존재하지 않습니다. 이것은 후위 감소 연산자(--)와 크다 비교 연산자(>)가 결합된 시각적 착시입니다.

코드를 실행하면 i가 0이 될 때까지 1씩 줄어들며 반복을 수행합니다. 띄어쓰기가 코드의 논리를 바꾸지 않는다는 것을 보여주는 흥미로운 트릭입니다.

```
static void Main(string[] args)
{
    int i = 10;
    while (i --> 0)
    {
        Console.WriteLine(i);
    }
}
```



while ((i--) > 0)